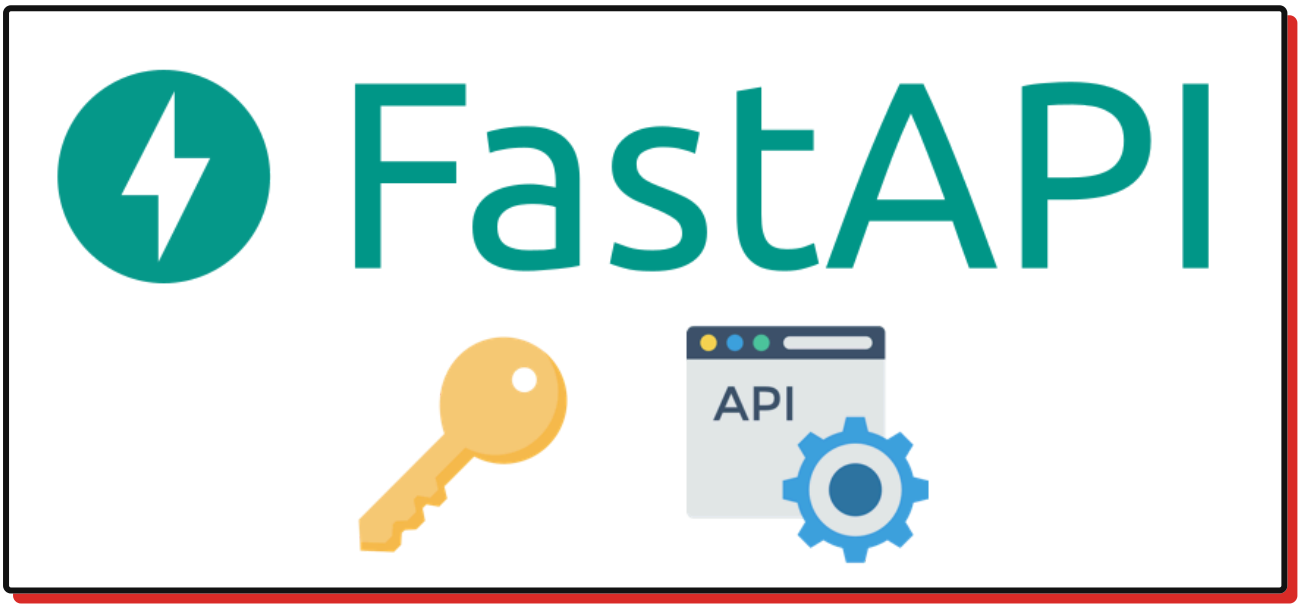
PROYECTO APIS Y ACCESO A BBDD: EXPLICACIÓN

Sergio Lozano Molina
2ºDAM



RESUMEN

En esta práctica lo que llevaremos a cabo será trabajar con **FastAPI** y **SQLite** para desarrollar una API REST de gestión de tareas. En este documento indicaremos dónde se ha guardado el desarrollo del proyecto en GitHub y responderemos a las preguntas teóricas sobre los conceptos principales del desarrollo de APIs y acceso a bases de datos (arquitectura REST, endpoints y métodos HTTP, parámetros de ruta y consulta, esquemas con Pydantic, códigos de estado, gestión de la conexión con `sqlite3`, inyección de dependencias con `Depends`, consultas parametrizadas y control de transacciones)

ÍNDICE

Tarea 1 Adjuntar el código en repositorio Github

Tarea 2 Responder a las siguientes preguntas

- Pregunta 1 API y REST
- Pregunta 2 Endpoints y métodos HTTP
- Pregunta 3 Parámetros de ruta y de consulta
- Pregunta 5 Códigos de estado HTTP
- Pregunta 6 SQLite y el módulo sqlite3
- Pregunta 7 La conexión a la base de datos
- Pregunta 8 Dependencias con Depends
- Pregunta 9 Consultas parametrizadas
- Pregunta 10 Confirmar cambios y detectar errores

TAREA 1: ADJUNTAR EL CÓDIGO EN REPOSITORIO GITHUB

El código del proyecto se encuentra en el siguiente enlace que lleva al repositorio de Github:
https://github.com/1oza16/Evaluaci-n-APIs-y-Acceso-a-BBDD.git

TAREA 2: RESPONDER A LAS SIGUIENTES PREGUNTAS

PREGUNTA 1: API Y REST

En esta pregunta analizaremos qué es una **API**, las características de una arquitectura **REST** y cómo interactúan las peticiones y respuestas HTTP entre el cliente y el servidor FastAPI.

Una **API** (Interfaz de Programación de Aplicaciones) es un conjunto de reglas y protocolos que permite a diferentes aplicaciones comunicarse e intercambiar información entre sí. Que sea **REST** (Representational State Transfer) significa que sigue un estilo de arquitectura software basado en el protocolo HTTP, utilizando sus métodos estándar (GET, POST, PUT, DELETE) para gestionar recursos de forma sin estado (***stateless***).

- **Petición HTTP (Cliente):** El cliente (un navegador web, la interfaz interactiva de Swagger UI o la herramienta de consola `curl`) envía una solicitud HTTP a la URL del servidor. Esta petición contiene un método HTTP, encabezados y opcionalmente un cuerpo (***body***) en formato JSON con los datos requeridos.
- **Procesamiento y Respuesta HTTP (Servidor FastAPI):** El servidor desarrollado en FastAPI recibe e interpreta la petición, ejecuta la lógica correspondiente (como consultar la base de datos SQLite) y devuelve una respuesta HTTP. Esta respuesta incluye un código de estado (como 200 OK o 201 Created), encabezados y un cuerpo de respuesta que contiene la información solicitada en formato JSON.

PREGUNTA 2: ENDPOINTS Y MÉTODOS HTTP

En este apartado explicaremos qué es un **endpoint** y repasaremos las operaciones disponibles en nuestra API de gestión de tareas.

Un **endpoint** es una ruta o URL específica de la API donde el servidor escucha y atiende las peticiones del cliente para llevar a cabo una acción concreta sobre un recurso.

- **Crear tarea (`POST /nueva/tarea`):** Inserta un nuevo registro en la base de datos SQLite a partir del cuerpo enviado por el cliente y devuelve la tarea creada junto con su nuevo `id` asignado y el código HTTP 201.
- **Listar tareas (`GET /tareas`):** Recupera y devuelve la lista completa de todas las tareas guardadas en la base de datos.
- **Obtener una tarea (`GET /tareas/{tarea_id}`):** Busca en la base de datos y devuelve únicamente la tarea que coincida con el identificador único `tarea_id` proporcionado.
- **Completar tarea (`PUT /tareas/{tarea_id}/completar`):** Actualiza el valor del campo `completada` a `1` en la base de datos para la tarea indicada y devuelve el registro actualizado.
- **Borrar tarea (`DELETE /tareas/borrar/{tarea_id}`):** Elimina permanentemente de la base de datos la tarea identificada por `tarea_id` y devuelve un código 204 sin contenido en el cuerpo.

PREGUNTA 3: PARÁMETROS DE RUTA Y DE CONSULTA

Aquí analizaremos la diferencia entre los **parámetros de ruta** y los **parámetros de consulta** (***query parameters***), viendo ejemplos concretos en nuestra API.

La diferencia principal consiste en cómo se estructuran dentro de la URL y cuál es su finalidad principal al realizar una petición HTTP.

- **Parámetro de ruta (`tarea_id`):** Forma parte obligatoria del camino de la URL (por ejemplo `/tareas/3`) y se emplea para identificar de forma exclusiva un recurso específico en el servidor.
- **Parámetro de consulta (`estado` / `completada`):** Se añade al final de la URL tras el carácter `?` (como `/tareas/filtradas?estado=true`) de forma opcional, con el fin de filtrar, ordenar o modificar los datos devueltos sin alterar la identidad del endpoint.
- **Detección en FastAPI:** FastAPI determina el tipo de parámetro examinando la firma de la función en Python. Si un parámetro está definido entre llaves dentro de la ruta del decorador (por ejemplo, `@app.get("/tareas/{tarea_id}")`), lo interpreta automáticamente como un **parámetro de ruta**. Si el parámetro no figura en la cadena de la ruta (como `estado: bool | None = None` en `buscar_tarea_filtrada`), FastAPI lo reconoce de inmediato como un **parámetro de consulta**.

PREGUNTA 4: ESQUEMAS CON PYDANTIC

En esta pregunta veremos la función de los esquemas declarados en el archivo `schemas.py` y cómo valida FastAPI la información de las peticiones.

Los esquemas creados mediante **Pydantic** sirven para definir la estructura de datos, los tipos de variables y las reglas de validación que debe cumplir la información recibida y devuelta por la aplicación.

- **Diferencia entre `TareaCrear` y `TareaLeer`:** La principal diferencia consiste en que `TareaCrear` se utiliza al recibir datos del cliente (Solicitudes `POST`), requiriendo únicamente los campos `titulo` y opcionalmente `descripcion`, mientras que `TareaLeer` define la estructura completa enviada en la respuesta al cliente, incluyendo además los campos `id` y `completada` asignados por la base de datos.
- `response_model`**:** Este especifica a FastAPI qué modelo de Pydantic debe aplicar para filtrar, validar y convertir los datos retornados por la función antes de ser serializados a JSON y enviados al cliente.
- **Cuerpo que incumple el esquema:** Si el cliente envía una petición con un cuerpo que no coincide con las restricciones del esquema (Por ejemplo si se omite el campo `titulo` o se pasa un entero donde se requiere una cadena de texto), provoca que FastAPI bloquea el procesamiento y genera de forma automática una respuesta de error **422 Unprocessable Entity**, detallando exactamente qué campo ha fallado.

PREGUNTA 5: CÓDIGOS DE ESTADO HTTP

A continuación repasaremos los códigos de estado HTTP más relevantes utilizados en nuestra API, qué significan y qué componente los genera.

Los códigos de estado HTTP indican el resultado final de la petición procesada por el servidor.

- **201 Created:** Significa que la petición se ha procesado correctamente y ha dado como resultado la creación de un nuevo recurso. Se utiliza en el endpoint `POST /nueva/tarea` y lo asigna FastAPI mediante el parámetro `status_code=201`.
- **204 No Content:** Indica que la solicitud ha finalizado con éxito, pero no requiere devolver ningún contenido en el cuerpo de la respuesta. Se emplea en `DELETE /tareas/borrar/{tarea_id}` y lo establece FastAPI mediante `status_code=status.HTTP_204_NO_CONTENT`.
- **404 Not Found:** Notifica que el recurso solicitado no existe en la base de datos. Se devuelve en los endpoints `GET /tareas/{tarea_id}`, `PUT /tareas/{tarea_id}/completar` y `DELETE /tareas/borrar/{tarea_id}` cuando la búsqueda en la base de datos no arroja ningún resultado. Lo generamos nosotros explícitamente lanzando una excepción `HTTPException(status_code=404, detail="...")`.
- **422 Unprocessable Entity:** Indica que la estructura o los datos del cuerpo de la petición son sintácticamente correctos pero no cumplen las validaciones esperadas por la aplicación. Aparece en cualquier endpoint que reciba datos de entrada cuando Pydantic detecta una incoherencia de tipos o la ausencia de campos obligatorios, y es generado automáticamente por FastAPI.

PREGUNTA 6: SQLITE Y EL MÓDULO SQLITE3

En este apartado analizaremos qué es **SQLite**, en qué se diferencia de un servidor como **PostgreSQL** y cómo se usa en nuestro proyecto.

SQLite consiste en un sistema de gestión de bases de datos relacional ligero y autocontenido que no requiere ejecutarse como un servicio o proceso servidor independiente.

- **Diferencia con PostgreSQL:** PostgreSQL funciona mediante un servidor independiente en el sistema el cual administra múltiples conexiones y requiere configuración de red y permisos. En cambio, SQLite se integra directamente dentro de la propia aplicación de Python, leyendo y escribiendo en un archivo local `.db`.
- **Ubicación de los datos:** En nuestra aplicación, todos los datos se almacenan físicamente en un único archivo llamado `tareas.db` en la raíz del proyecto.
- **Módulo `sqlite3`:** Esta es la librería nativa de Python que nos proporciona las funciones para conectar con `tareas.db`, ejecutar consultas SQL y gestionar las transacciones..

PREGUNTA 7: LA CONEXIÓN A LA BASE DE DATOS

Aquí explicaremos la función de `check_same_thread=False` y `row_factory = sqlite3.Row` dentro de `get_connection()` en `database.py`.
Un ejemplo de esta función en el código es:

```
def get_connection():
    conn = sqlite3.connect(DB_FILE, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    return conn
```

- **check_same_thread=False:** Por defecto, SQLite impide que una misma conexión sea utilizada en distintos hilos de ejecución. Desactivar esta opción es imprescindible en **FastAPI**, ya que el framework atiende múltiples peticiones concurrentes utilizando varios hilos.
- **row_factory = sqlite3.Row:** Modifica la forma en que SQLite devuelve las filas de los resultados. Permite acceder a las columnas por su nombre en lugar de por índice y convertir las filas directamente a diccionarios mediante `dict(fila)`, facilitando su conversión a JSON.

PREGUNTA 8: DEPENDENCIAS CON DEPENDS

En esta pregunta analizaremos el funcionamiento de `Depends(get_db)` y cómo administra el ciclo de vida de la conexión a la base de datos.

El uso de `Depends(get_db)` permite inyectar una conexión activa en cada endpoint de forma automática y centralizada.
El codigo usado en `get_db()` es el siguiente, el cual lo analizaremos:

```python
def get_db():
    db = get_connection()
    try:
        yield db
    finally:
        db.close()
```

- **Conexión por petición:** Esta función permite abrir y cerrar una conexión independiente en cada solicitud evita bloqueos prolongados y garantiza el aislamiento total de las operaciones entre usuarios.
- **Funcion de `yield`:** Este pausa la ejecución de la función generadora tras crear la conexión, entregando el objeto `db` al endpoint para que pueda realizar las operaciones SQL.
- **Uso de `finally`:** Asegura que el bloque `db.close()` se ejecute siempre al finalizar el procesamiento de la petición (incluso si ocurre un error o una excepción `HTTPException`), impidiendo cualquier fuga de conexiones.

PREGUNTA 9: CONSULTAS PARAMETRIZADAS

En este apartado explicaremos la importancia de utilizar **consultas parametrizadas** y los riesgos de la inyección de código SQL.

En el código escribimos las consultas utilizando el carácter `?` como marcador de posición y pasando los valores en una tupla independiente (por ejemplo `db.execute("SELECT * FROM tareas WHERE id = ?", (tarea_id,))`).

- **Riesgo de Inyección SQL:** Si construyéramos la consulta mediante concatenación de cadenas (Como `"SELECT * FROM tareas WHERE id = "` `+ tarea_id`), un usuario malintencionado podría enviar código SQL en el parámetro (Por ejemplo `1 OR 1=1` o `1; DROP TABLE tareas`), alterando la instrucción recibida por la base de datos y comprometiendo toda la información.
- **Tratamiento seguro de datos:** Al utilizar consultas parametrizadas, el motor de SQLite interpreta los valores recibidos estrictamente como datos literales y jamás como instrucciones de código, anulando cualquier intento de inyección SQL.

PREGUNTA 10: CONFIRMAR CAMBIOS Y DETECTAR ERRORES

Por último, explicaremos para qué sirve `db.commit()` y cómo se utiliza `cursor.rowcount` para manejar los errores cuando un recurso no existe.

Al realizar modificaciones en la base de datos es imprescindible asegurar la persistencia y comprobar el resultado de las operaciones.

- **Uso de `db.commit()`:** Sentencias como `INSERT`, `UPDATE` o `DELETE` se ejecutan dentro de una transacción temporal. Es necesario llamar a `db.commit()` para guardar definitivamente los cambios en el archivo `.db`; si no se hiciera, las modificaciones se descartarían al cerrar la conexión.
- **Uso de `cursor.rowcount`:** Es una propiedad del cursor que devuelve el número exacto de filas afectadas por la última instrucción SQL ejecutada.
- **Detección de 404:** Si al ejecutar una actualización o borrado `cursor.rowcount == 0`, significa que ninguna fila coincidía con la condición (el `tarea_id` no existía). Al detectar este caso, lanzamos la excepción `HTTPException(status_code=404, detail="Tarea no encontrada")` para notificar al cliente.